

# Data Structures & Algorithms

A Comprehensive Learning Guide  
Basic to Advanced

---

Arrays | Linked Lists | Stacks | Queues  
Trees | Graphs | Hash Tables | Heaps  
Sorting | Searching | Dynamic Programming  
Greedy Algorithms | Backtracking  
Practice Problems with Solutions

*Theory | Concepts | Examples | Visual Explanations*

## Table of Contents

---

|     |                              |    |
|-----|------------------------------|----|
| 1.  | Introduction to DSA          | 3  |
| 2.  | Complexity Analysis          | 3  |
| 3.  | Arrays                       | 5  |
| 4.  | Strings                      | 8  |
| 5.  | Linked Lists                 | 10 |
| 6.  | Stacks                       | 13 |
| 7.  | Queues                       | 15 |
| 8.  | Hash Tables (Hashing)        | 17 |
| 9.  | Trees                        | 20 |
| 10. | Binary Search Trees (BST)    | 23 |
| 11. | Heaps & Priority Queues      | 25 |
| 12. | Graphs                       | 27 |
| 13. | Graph Algorithms             | 30 |
| 14. | Sorting Algorithms           | 33 |
| 15. | Searching Algorithms         | 37 |
| 16. | Dynamic Programming          | 39 |
| 17. | Greedy Algorithms            | 43 |
| 18. | Backtracking                 | 45 |
| 19. | Trie (Prefix Tree)           | 47 |
| 20. | Union-Find (Disjoint Set)    | 48 |
| 21. | Practice Problems Collection | 50 |

# 1. Introduction to Data Structures & Algorithms

Data Structures and Algorithms (DSA) form the backbone of computer science and software engineering. They are essential for writing efficient, optimized, and scalable code.

## What is a Data Structure?

A data structure is a way of organizing, storing, and managing data in a computer so that it can be accessed and modified efficiently. Different data structures are suited for different kinds of tasks, and some are highly specialized for specific purposes.

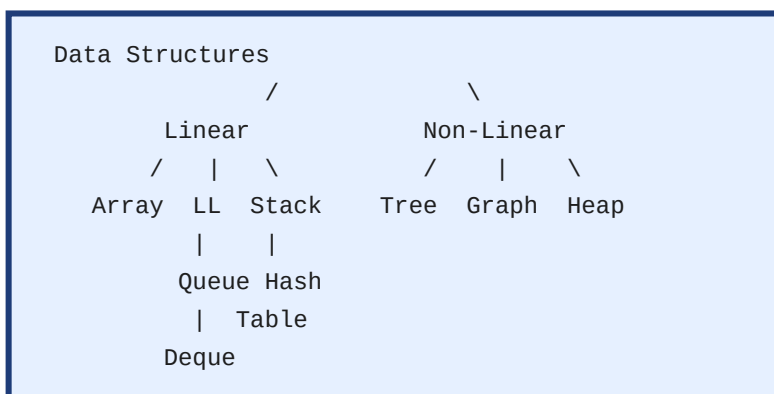
## What is an Algorithm?

An algorithm is a step-by-step procedure or set of rules to be followed in calculations or other problem-solving operations. A good algorithm is efficient, clear, and generalizable.

## Why Learn DSA?

1. Technical Interviews: DSA is the most tested topic in coding interviews at top tech companies.
2. Problem Solving: Develops analytical and logical thinking.
3. Efficiency: Understanding DSA helps write code that runs faster and uses less memory.
4. Foundation: Essential for advanced topics like system design, databases, and OS.
5. Career Growth: Opens doors to better job opportunities and higher salaries.

## Classification of Data Structures



## Classification of Algorithms

## Algorithms

|         |         |           |       |  |
|---------|---------|-----------|-------|--|
|         | /       |           | \     |  |
|         | Sorting | Searching | Graph |  |
| /     \ |         |           | /   \ |  |
| B S M Q | Linear  | BFS       | DFS   |  |
| u e e u | Binary  |           |       |  |
| b l r i | Search  |           |       |  |
| b e g   |         |           |       |  |
| l c o   |         |           |       |  |
| e t r   |         |           |       |  |
| i       |         |           |       |  |
| o       |         |           |       |  |
| n       |         |           |       |  |

## 2. Complexity Analysis

Complexity analysis helps us understand how the runtime or memory usage of an algorithm grows as the input size grows. This is fundamental to writing efficient code.

### Time Complexity

Time complexity measures the amount of time an algorithm takes to complete as a function of the input size. We use Big O notation to describe the upper bound of the growth rate.

### Common Time Complexities

| Complexity    | Name         | Example                      |
|---------------|--------------|------------------------------|
| $O(1)$        | Constant     | Array access by index        |
| $O(\log n)$   | Logarithmic  | Binary search                |
| $O(n)$        | Linear       | Linear search, single loop   |
| $O(n \log n)$ | Linearithmic | Merge sort, Quick sort (avg) |
| $O(n^2)$      | Quadratic    | Bubble sort, nested loops    |
| $O(n^3)$      | Cubic        | Matrix multiplication        |
| $O(2^n)$      | Exponential  | Subset generation            |
| $O(n!)$       | Factorial    | Permutation generation       |

### Visual Comparison of Growth Rates

| n   | $O(1)$ | $O(\log n)$ | $O(n)$ | $O(n \log n)$ | $O(n^2)$ | $O(2^n)$ |
|-----|--------|-------------|--------|---------------|----------|----------|
| 1   | 1      | 0           | 1      | 0             | 1        | 2        |
| 4   | 1      | 2           | 4      | 8             | 16       | 16       |
| 8   | 1      | 3           | 8      | 24            | 64       | 256      |
| 16  | 1      | 4           | 16     | 64            | 256      | 65536    |
| 32  | 1      | 5           | 32     | 160           | 1024     | ~4B      |
| 100 | 1      | 7           | 100    | 700           | 10000    | huge     |

### Space Complexity

Space complexity measures the amount of memory an algorithm uses relative to the input size. It includes both auxiliary space (extra space used) and input space.

#### Big O Notation (Formal Definition)

Big O notation describes the UPPER BOUND of an algorithm's growth rate.

$f(n) = O(g(n))$  if there exist positive constants  $c$  and  $n_0$  such that

$f(n) \leq c * g(n)$  for all  $n \geq n_0$ .

This means: For large enough inputs,  $g(n)$  is always an upper bound on  $f(n)$  (up to a constant factor).

## How to Calculate Time Complexity

Rules for analyzing time complexity:

1. Ignore constants:  $O(2n)$  becomes  $O(n)$
2. Take the dominant term:  $O(n^2 + n)$  becomes  $O(n^2)$
3. Loops: A single loop over  $n$  elements =  $O(n)$
4. Nested loops: Two nested loops =  $O(n^2)$
5. Sequential statements: Add complexities
6. If-else: Take the maximum of both branches

```
// O(n) - Single loop
for (int i = 0; i < n; i++)
    count++;

// O(n^2) - Nested loop
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        count++;

// O(log n) - Divide by 2 each time
while (n > 1)
    n = n / 2;

// O(sqrt(n))
for (int i = 1; i * i <= n; i++)
    count++;
```

## 3. Arrays

An array is a collection of elements stored at contiguous memory locations. It is the simplest and most widely used data structure. Elements can be accessed directly using their index.

### Array Representation

```

Index:    0      1      2      3      4
          +-----+-----+-----+-----+
Value:    | 10   | 20   | 30   | 40   | 50   |
          +-----+-----+-----+-----+
Memory: 1000  1004  1008  1012  1016
(Each int takes 4 bytes)

```

### Key Properties

1. Fixed Size: Size must be known at declaration (in most languages)
2. Contiguous Memory: Elements are stored side by side
3. Direct Access: Access any element in  $O(1)$  using index
4. Homogeneous: All elements must be of the same type
5. Index starts at 0 in most languages (0-based indexing)

### Time Complexity of Array Operations

| Operation         | Time Complexity | Description                        |
|-------------------|-----------------|------------------------------------|
| Access (by index) | $O(1)$          | Direct access using memory address |
| Search (unsorted) | $O(n)$          | Linear search through array        |
| Search (sorted)   | $O(\log n)$     | Binary search                      |
| Insert (at end)   | $O(1)$          | Amortized for dynamic arrays       |
| Insert (at start) | $O(n)$          | Shift all elements right           |
| Insert (at index) | $O(n)$          | Shift elements and insert          |
| Delete (at end)   | $O(1)$          | Remove last element                |
| Delete (at start) | $O(n)$          | Shift all elements left            |

### Example: Find Maximum Element

```
int findMax(int arr[], int n) {
    int maxVal = arr[0];           // O(1)
    for (int i = 1; i < n; i++)    // O(n)
        if (arr[i] > maxVal)
            maxVal = arr[i];       // O(1)
    return maxVal;                 // Total: O(n)
}
```

## Example: Reverse an Array

```
void reverse(int arr[], int n) {
    int left = 0, right = n - 1;
    while (left < right) {
        int temp = arr[left];
        arr[left] = arr[right];
        arr[right] = temp;
        left++;
        right--;
    }
    // Time: O(n), Space: O(1)
}
```

Before: [1, 2, 3, 4, 5]  
 Step 1: [5, 2, 3, 4, 1] swap(0,4)  
 Step 2: [5, 4, 3, 2, 1] swap(1,3)  
 Left meets Right -> Done!

## Two Pointer Technique (Array)

The two pointer technique uses two indices to traverse an array, often from opposite ends. It reduces  $O(n^2)$  brute force solutions to  $O(n)$ .

```
// Check if array is a palindrome
bool isPalindrome(int arr[], int n) {
    int left = 0, right = n - 1;
    while (left < right) {
        if (arr[left] != arr[right])
            return false;
        left++;
        right--;
    }
    return true;
}
```

## Sliding Window Technique

Sliding window is used for problems involving contiguous subarrays of fixed size  $k$ . Instead of recalculating from scratch each time, we slide the window.

```
// Maximum sum of k consecutive elements
int maxSum(int arr[], int n, int k) {
    int windowSum = 0;
    // Calculate sum of first window
    for (int i = 0; i < k; i++)
        windowSum += arr[i];

    int maxSum = windowSum;
    // Slide the window
    for (int i = k; i < n; i++) {
        windowSum += arr[i] - arr[i - k];
        maxSum = max(maxSum, windowSum);
    }
    return maxSum;
    // Time: O(n), Space: O(1)
}
```

## 4. Strings

A string is a sequence of characters. In most languages, strings are immutable (Java, Python), meaning each modification creates a new string. Understanding string manipulation is crucial for interviews.

### String as an Array

```
String: "HELLO"
Index:   0  1  2  3  4
Char:    'H' 'E' 'L' 'L' 'O'
ASCII:    72 69 76 76 79
```

### Common String Operations & Their Complexity

| Operation        | Time     | Notes                     |
|------------------|----------|---------------------------|
| Access by index  | $O(1)$   | Same as array             |
| Search substring | $O(n*m)$ | $n$ = text, $m$ = pattern |
| Concatenation    | $O(n+m)$ | Creates new string        |
| Check palindrome | $O(n)$   | Two pointer approach      |
| Check anagram    | $O(n)$   | Character frequency count |

### Example: Reverse a String

```
// Python
def reverse_string(s):
    return s[::-1] # O(n) time, O(n) space

// Two-pointer approach (in-place for char array)
def reverse_string(s):
    left, right = 0, len(s) - 1
    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1
```

### Example: Check if Two Strings are Anagrams

Two strings are anagrams if they contain the same characters with the same frequencies.

```
def is_anagram(s1, s2):
    if len(s1) != len(s2):
        return False
    # Method 1: Sorting - O(n log n)
    return sorted(s1) == sorted(s2)

def is_anagram(s1, s2):
    # Method 2: Frequency count - O(n)
    freq = [0] * 26
    for c in s1:
        freq[ord(c) - ord('a')] += 1
    for c in s2:
        freq[ord(c) - ord('a')] -= 1
    return all(f == 0 for f in freq)
```

## KMP Pattern Matching (Advanced)

The Knuth-Morris-Pratt (KMP) algorithm finds a pattern in a text in  $O(n+m)$  time, much better than the naive  $O(n*m)$ . It uses a 'lps' (longest proper prefix which is also suffix) array.

```
def kmp_search(text, pattern):
    # Step 1: Build LPS array
    lps = [0] * len(pattern)
    length, i = 0, 1
    while i < len(pattern):
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1
        elif length != 0:
            length = lps[length - 1]
        else:
            lps[i] = 0
            i += 1
    # Step 2: Search
    i = j = 0
    while i < len(text):
        if text[i] == pattern[j]:
            i += 1; j += 1
        if j == len(pattern):
            print(f'Pattern found at {i - j}')
            j = lps[j - 1]
        elif i < len(text) and text[i] != pattern[j]:
            if j != 0: j = lps[j - 1]
            else: i += 1
```

## Types of Linked Lists

```
* = next pointer, / = NULL
```

## Node Structure

## Time Complexity

Page 12/59

## Example: Insert at Beginning

```
def insert_at_head(head, data):
    new_node = Node(data)
    new_node.next = head # Point new node to old head
    return new_node      # New node becomes head
# Time: O(1), Space: O(1)
```

Insert 5 at head of [1]->[2]->[3]->NULL

Step 1: Create [5]  
 Step 2: [5]->[1]->[2]->[3]->NULL  
 Step 3: head = [5]

## Example: Delete a Node by Value

```
def delete_node(head, key):
    # If head needs to be deleted
    if head and head.data == key:
        return head.next
    current = head
    while current and current.next:
        if current.next.data == key:
            current.next = current.next.next
            return head
        current = current.next
    return head
# Time: O(n), Space: O(1)
```

## Detect Cycle (Floyd's Algorithm)

Floyd's Cycle Detection uses two pointers (slow and fast). If they meet, there's a cycle. This is also called the 'Tortoise and Hare' algorithm.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next      # moves 1 step
        fast = fast.next.next # moves 2 steps
        if slow == fast:
            return True # cycle detected!
    return False
# Time: O(n), Space: O(1)
```

No cycle: [1]->[2]->[3]->[4]->NULL

Fast reaches NULL -> No cycle

Cycle: [1]->[2]->[3]->[4]

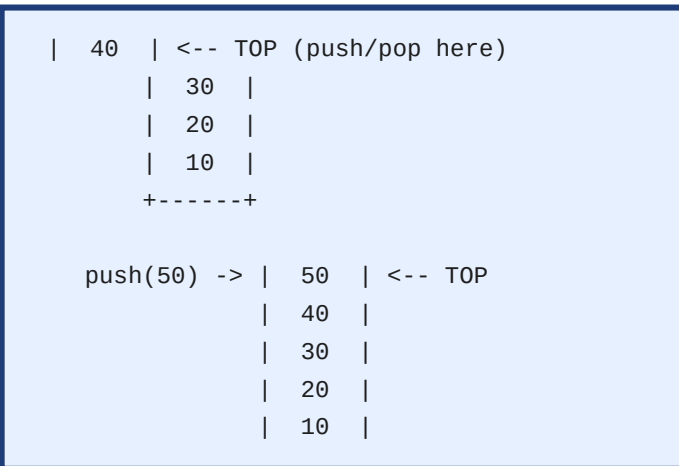
^            |  
+-----<-----+

Fast and Slow meet at 3 -> Cycle!

## 6. Stacks

A stack is a Last-In-First-Out (LIFO) data structure. The last element added is the first one removed. Think of it as a stack of plates - you add and remove from the top only.

### Stack Operations



### Time Complexity

| Operation    | Time | Description                       |
|--------------|------|-----------------------------------|
| push(x)      | O(1) | Add element to top                |
| pop()        | O(1) | Remove element from top           |
| top()/peek() | O(1) | View top element without removing |
| isEmpty()    | O(1) | Check if stack is empty           |
| size()       | O(1) | Get number of elements            |
| Search(x)    | O(n) | Search for element in stack       |

### Implementation

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if self.is_empty():
            raise IndexError('Stack is empty')
        return self.items.pop()

    def peek(self):
        return self.items[-1] if self.items else None

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)
```

## Applications of Stack

1. Undo/Redo operations in text editors
2. Browser back/forward navigation
3. Function call stack (recursion)
4. Expression evaluation (infix to postfix)
5. Balanced parentheses validation
6. Tower of Hanoi
7. DFS graph traversal

## Example: Balanced Parentheses

```
def is_balanced(s):
    stack = []
    mapping = {'(': ')', '[': ']', '{': '}'
    for char in s:
        if char in mapping.values(): # opening
            stack.append(char)
        elif char in mapping: # closing
            if not stack or stack[-1] != mapping[char]:
                return False
            stack.pop()
    return len(stack) == 0

print(is_balanced('{([])}')) # True
print(is_balanced('{[()] }')) # False
```

## Example: Next Greater Element

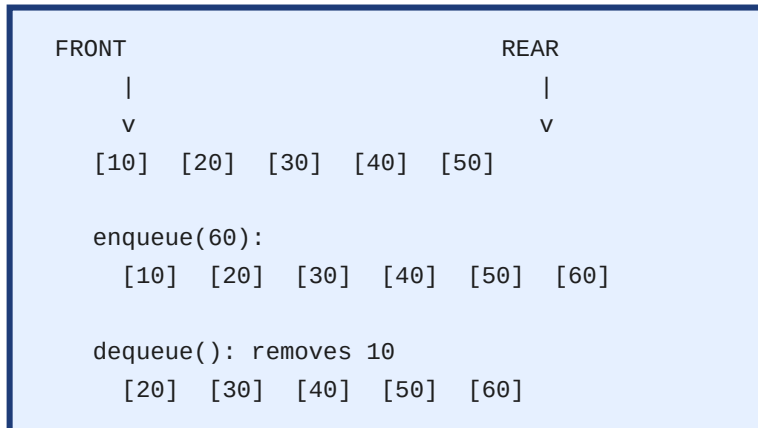
```
def next_greater(arr):
    n = len(arr)
    result = [-1] * n
    stack = [] # stores indices
    for i in range(n):
        while stack and arr[i] > arr[stack[-1]]:
            result[stack.pop()] = arr[i]
        stack.append(i)
    return result

# Input:  [4, 5, 2, 25]
# Output: [5, 25, 25, -1]
# Time: O(n), Space: O(n)
```

## 7. Queues

A queue is a First-In-First-Out (FIFO) data structure. Elements are added at the rear (enqueue) and removed from the front (dequeue). Think of it as a line at a ticket counter.

### Queue Operations



### Types of Queues

1. Simple Queue: Basic FIFO queue
2. Circular Queue: Connects end to beginning, efficient use of space
3. Deque (Double-ended Queue): Insert/remove from both ends
4. Priority Queue: Elements have priorities, highest priority removed first

### Time Complexity

| Operation      | Simple Queue | Circular Queue | Deque  |
|----------------|--------------|----------------|--------|
| Enqueue/Insert | $O(1)^*$     | $O(1)$         | $O(1)$ |
| Dequeue/Remove | $O(1)^*$     | $O(1)$         | $O(1)$ |
| Front/Ppeek    | $O(1)$       | $O(1)$         | $O(1)$ |
| Search         | $O(n)$       | $O(n)$         | $O(n)$ |

### Implementation Using Two Stacks

```
class QueueUsingStacks:
    def __init__(self):
        self.stack_in = [] # for enqueue
        self.stack_out = [] # for dequeue

    def enqueue(self, x):
        self.stack_in.append(x)

    def dequeue(self):
        if not self.stack_out:
            while self.stack_in:
                self.stack_out.append(self.stack_in.pop())
        if not self.stack_out:
            raise IndexError('Queue is empty')
        return self.stack_out.pop()
```

## Applications of Queue

1. BFS graph/tree traversal
2. CPU scheduling (Round Robin)
3. Print queue (jobs in order)
4. Buffering (IO buffers, streaming)
5. Sliding window problems
6. Level-order traversal of trees

## 8. Hash Tables (Hashing)

A hash table maps keys to values using a hash function. It provides near  $O(1)$  average time for insertions, deletions, and lookups, making it one of the most important data structures.

### How Hashing Works

```
Key: "apple" -> hash("apple") -> 7
Key: "banana" -> hash("banana") -> 2

Hash Table (array of buckets):
Index:  0   1   2       3   4   5   6   7
        [---|---|"banana"|---|---|---|---|"apple"]
```

### Collision Handling

#### 1. Chaining (Open Hashing)

```
Index 0: -> ["cat", 5] -> ["hat", 8] -> NULL
Index 1: -> NULL
Index 2: -> ["banana", 3] -> NULL
Index 3: -> ["dog", 7] -> ["log", 2] -> NULL
(Each bucket is a linked list)
```

#### 2. Open Addressing (Closed Hashing)

Linear Probing: If index is taken, try (index+1), (index+2), etc.

Quadratic Probing: Try (index+1<sup>2</sup>), (index+2<sup>2</sup>), etc.

Double Hashing: Use a second hash function for step size.

```
Linear Probing Example:
hash("apple") = 3 -> Table[3] = "apple"
hash("grape") = 3 -> Collision!
-> Table[4] = "grape" (try next slot)
```

### Load Factor

Load Factor ( $\alpha$ ) =  $n / k$  where  $n$  = number of elements,  $k$  = number of buckets

When  $\alpha$  exceeds a threshold (typically 0.75), the table is resized (rehashed).

New table is typically double the size, and all elements are re-inserted.

This maintains  $O(1)$  average time by keeping collisions rare.

### Time Complexity

| Operation | Average | Worst Case |
|-----------|---------|------------|
|-----------|---------|------------|

|        |        |                                  |
|--------|--------|----------------------------------|
| Insert | $O(1)$ | $O(n)$ - all hash to same bucket |
| Search | $O(1)$ | $O(n)$                           |
| Delete | $O(1)$ | $O(n)$                           |

## Example: Two Sum Problem

```
def two_sum(nums, target):
    seen = {} # hash map
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

# nums = [2, 7, 11, 15], target = 9
# Output: [0, 1] (because 2 + 7 = 9)
# Time:  $O(n)$ , Space:  $O(n)$ 
```

## Frequency Counting Pattern

```
from collections import Counter

def top_k_frequent(nums, k):
    count = Counter(nums) #  $O(n)$ 
    return [x for x, _ in count.most_common(k)]

# Or using a hash map manually:
def top_k_frequent(nums, k):
    freq = {}
    for num in nums:
        freq[num] = freq.get(num, 0) + 1
    return sorted(freq, key=freq.get, reverse=True)[:k]
```

## 9. Trees

A tree is a hierarchical data structure consisting of nodes connected by edges. It has a root node at the top, and each node has zero or more children.

### Tree Terminology

```

100          <- Root
  /  \
 50   150    <- Children of 100
 /  \  /  \
25   75 125 200 <- Leaves (no children)

```

Root: 100 (top node)

Parent of 50: 100

Children of 50: 25, 75

Leaf nodes: 25, 75, 125, 200

Height: 2 (edges from root to deepest leaf)

Depth/Level: root=0, 50=1, 25=2

### Types of Trees

**BINARY TREE:** Each node has at most 2 children

```

  1
 / \
2   3
 / \
4   5

```

**FULL BINARY TREE:** Every node has 0 or 2 children

```

  1
 / \
2   3
 / \
4   5

```

**COMPLETE BINARY TREE:** All levels full except last (filled left to right)

```

  1
 / \
2   3
 / \ /
4  5 6

```

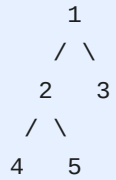
### Tree Traversals

There are two main categories of tree traversal:

1. Depth-First Search (DFS): Goes deep before going wide
2. Breadth-First Search (BFS): Goes wide before going deep (Level Order)

### DFS Traversals (3 types)

Tree for all traversals:



Inorder (Left, Root, Right): 4 2 5 1 3

Preorder (Root, Left, Right): 1 2 4 5 3

Postorder (Left, Right, Root): 4 5 2 3 1

```
# Binary Tree Node
class TreeNode:
    def __init__(self, val=0):
        self.val = val
        self.left = None
        self.right = None

# Inorder: Left -> Root -> Right
def inorder(root):
    if root:
        inorder(root.left)
        print(root.val)
        inorder(root.right)

# Preorder: Root -> Left -> Right
def preorder(root):
    if root:
        print(root.val)
        preorder(root.left)
        preorder(root.right)

# Postorder: Left -> Right -> Root
def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.val)

# Level Order (BFS)
from collections import deque
def level_order(root):
    if not root: return []
    result, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)
        result.append(level)
    return result
```

## Height and Size of Tree

```
# Height of tree (number of edges on longest path)
def height(root):
    if root is None:
        return -1
    return 1 + max(height(root.left), height(root.right))

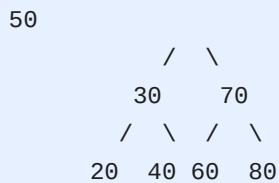
# Number of nodes in tree
def size(root):
    if root is None:
        return 0
    return 1 + size(root.left) + size(root.right)

# Check if two trees are identical
def is_same(p, q):
    if not p and not q: return True
    if not p or not q: return False
    return (p.val == q.val and
            is_same(p.left, q.left) and
            is_same(p.right, q.right))
```

## 10. Binary Search Trees (BST)

A Binary Search Tree is a binary tree with a special property: for every node, all values in the left subtree are smaller, and all values in the right subtree are larger. This enables efficient searching.

### BST Property



Rule: left < root < right (for every node)

Inorder traversal gives sorted order: 20 30 40 50 60 70 80

### BST Operations

| Operation         | Average     | Worst (skewed) |
|-------------------|-------------|----------------|
| Search            | $O(\log n)$ | $O(n)$         |
| Insert            | $O(\log n)$ | $O(n)$         |
| Delete            | $O(\log n)$ | $O(n)$         |
| Find Min/Max      | $O(\log n)$ | $O(n)$         |
| Inorder Successor | $O(\log n)$ | $O(n)$         |

### BST Implementation

```

class BST:
    def __init__(self):
        self.root = None

    def insert(self, val):
        self.root = self._insert(self.root, val)

    def _insert(self, node, val):
        if not node:
            return TreeNode(val)
        if val < node.val:
            node.left = self._insert(node.left, val)
        elif val > node.val:
            node.right = self._insert(node.right, val)
        return node

    def search(self, val):
        return self._search(self.root, val)

    def _search(self, node, val):
        if not node or node.val == val:
            return node
        if val < node.val:
            return self._search(node.left, val)
        return self._search(node.right, val)

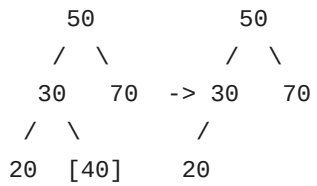
    def delete(self, val):
        self.root = self._delete(self.root, val)

    def _delete(self, node, val):
        if not node:
            return node
        if val < node.val:
            node.left = self._delete(node.left, val)
        elif val > node.val:
            node.right = self._delete(node.right, val)
        else: # found the node
            # Case 1 & 2: no child or one child
            if not node.left:
                return node.right
            if not node.right:
                return node.left
            # Case 3: two children
            # Replace with inorder successor (min of right subtree)
            successor = self._min_node(node.right)
            node.val = successor.val
            node.right = self._delete(node.right, successor.val)
        return node

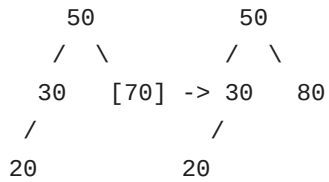
```

## Visual: BST Deletion Cases

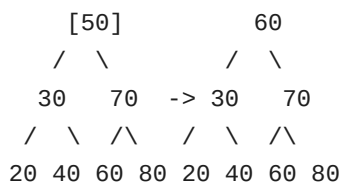
Case 1: Delete leaf (40) - just remove



Case 2: Delete node with one child (70)



Case 3: Delete node with two children (50)

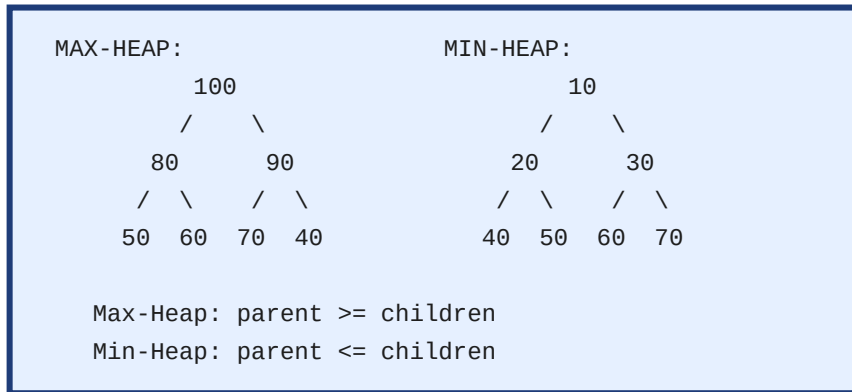


(Replace with inorder successor: 60)

## 11. Heaps & Priority Queues

A heap is a complete binary tree that satisfies the heap property. In a Max-Heap, every parent is  $\geq$  its children. In a Min-Heap, every parent is  $\leq$  its children.

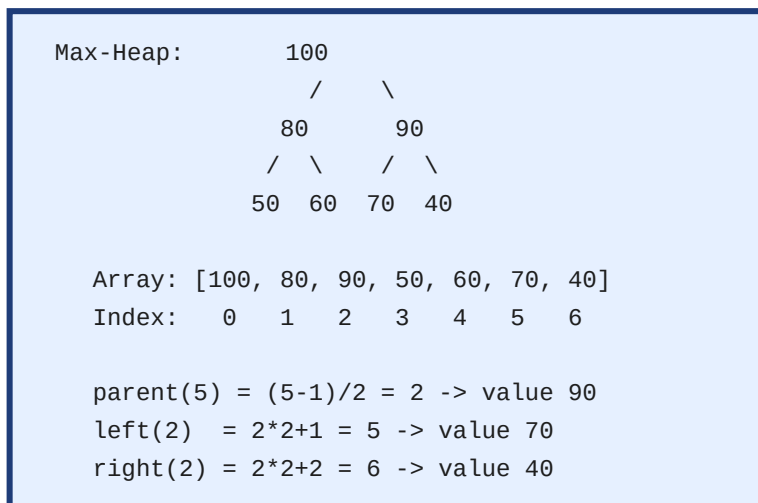
### Max-Heap vs Min-Heap



### Array Representation of Heap

Heaps are typically stored as arrays. For a node at index  $i$ :

- Parent:  $(i-1)/2$
- Left child:  $2*i + 1$
- Right child:  $2*i + 2$



### Heap Operations

| Operation       | Time          | Description           |
|-----------------|---------------|-----------------------|
| Get Min/Max     | $O(1)$        | Root element          |
| Insert          | $O(\log n)$   | Add at end, bubble up |
| Extract Min/Max | $O(\log n)$   | Remove root, heapify  |
| Build Heap      | $O(n)$        | From unsorted array   |
| Heap Sort       | $O(n \log n)$ | Repeated extraction   |

## Example: Top K Elements

Heaps are perfect for finding the K largest/smallest elements efficiently.

```
import heapq

# Find K largest elements - O(n log k)
def top_k_largest(nums, k):
    return heapq.nlargest(k, nums)

# Find K smallest elements - O(n log k)
def top_k_smallest(nums, k):
    return heapq.nsmallest(k, nums)

# Find Kth largest element
def kth_largest(nums, k):
    min_heap = []
    for num in nums:
        heapq.heappush(min_heap, num)
        if len(min_heap) > k:
            heapq.heappop(min_heap)
    return min_heap[0]
# Time: O(n log k), Space: O(k)
```

## Priority Queue Applications

1. Dijkstra's shortest path algorithm
2. Task scheduling (OS processes)
3. Merge K sorted lists/arrays
4. Median of a data stream
5. Find Kth largest element
6. Huffman encoding (compression)

## 12. Graphs

A graph is a collection of vertices (nodes) and edges (connections between nodes). Graphs are used to model relationships between objects - social networks, maps, networks, etc.

### Graph Types

|                                                                                       |                                                                                  |
|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>UNDIRECTED:</b><br><pre> A --- B     /       /   C --- D </pre>                    | <b>DIRECTED (Digraph):</b><br><pre> A --&gt; B         v     v C --&gt; D </pre> |
| <b>WEIGHTED:</b><br><pre> A --5-- B         /  3       4 2     /     C --6-- D </pre> | <b>UNWEIGHTED:</b><br><pre> A --- B     /       /   C --- D </pre>               |

### Graph Representations

#### 1. Adjacency Matrix

|                                      |              |   |   |   |
|--------------------------------------|--------------|---|---|---|
| Graph: A-B, A-C, B-C, B-D, C-D       |              |   |   |   |
| Matrix:                              | A            | B | C | D |
| A:                                   | [0, 1, 1, 0] |   |   |   |
| B:                                   | [1, 0, 1, 1] |   |   |   |
| C:                                   | [1, 1, 0, 1] |   |   |   |
| D:                                   | [0, 1, 1, 0] |   |   |   |
| Space: $O(V^2)$ , Check edge: $O(1)$ |              |   |   |   |

#### 2. Adjacency List

|                                                  |
|--------------------------------------------------|
| A -> [B, C]                                      |
| B -> [A, C, D]                                   |
| C -> [A, B, D]                                   |
| D -> [B, C]                                      |
| Space: $O(V+E)$ , Check edge: $O(\text{degree})$ |

### Comparison

| Feature | Adj Matrix | Adj List |
|---------|------------|----------|
| Space   | $O(V^2)$   | $O(V+E)$ |

|                |              |                    |
|----------------|--------------|--------------------|
| Check Edge     | $O(1)$       | $O(\text{degree})$ |
| List Neighbors | $O(V)$       | $O(\text{degree})$ |
| Add Edge       | $O(1)$       | $O(1)$             |
| Best For       | Dense graphs | Sparse graphs      |

## Graph Implementation

```
# Adjacency List using Dictionary
class Graph:
    def __init__(self):
        self.adj = {}

    def add_vertex(self, v):
        if v not in self.adj:
            self.adj[v] = []

    def add_edge(self, u, v, directed=False):
        self.add_vertex(u)
        self.add_vertex(v)
        self.adj[u].append(v)
        if not directed:
            self.adj[v].append(u)

    def bfs(self, start):
        visited = set()
        queue = deque([start])
        visited.add(start)
        order = []
        while queue:
            node = queue.popleft()
            order.append(node)
            for neighbor in self.adj[node]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    queue.append(neighbor)
        return order

    def dfs(self, start, visited=None):
        if visited is None:
            visited = set()
        visited.add(start)
        order = [start]
        for neighbor in self.adj[start]:
            if neighbor not in visited:
                order.extend(self.dfs(neighbor, visited))
        return order
```

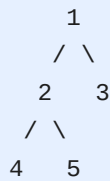
## 13. Graph Algorithms

### BFS vs DFS

**BFS (Breadth-First Search):**

Explores all neighbors first, then moves to next level.

Uses a QUEUE.



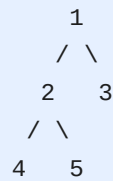
BFS order: 1,2,3,4,5

Level by level

**DFS (Depth-First Search):**

Goes deep along each branch

Uses a STACK or recursion.



DFS order: 1,2,4,5,3

Deep first, then backtrack

### Dijkstra's Shortest Path Algorithm

Finds the shortest path from a source vertex to all other vertices in a weighted graph with non-negative edge weights. Uses a min-heap (priority queue).

```

import heapq

def dijkstra(graph, source):
    dist = {v: float('inf') for v in graph}
    dist[source] = 0
    pq = [(0, source)] # (distance, vertex)

    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue # stale entry
        for v, weight in graph[u]:
            if dist[u] + weight < dist[v]:
                dist[v] = dist[u] + weight
                heapq.heappush(pq, (dist[v], v))
    return dist

# Time: O((V+E) log V) with min-heap
  
```

### Bellman-Ford Algorithm

Handles negative edge weights (unlike Dijkstra). Detects negative cycles. Relaxes all edges V-1 times. If any distance can still be reduced, negative cycle exists.

```
def bellman_ford(V, edges, source):
    dist = [float('inf')] * V
    dist[source] = 0

    # Relax all edges V-1 times
    for _ in range(V - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w

    # Check for negative cycles
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            print('Negative cycle detected!')
            return None

    return dist
# Time: O(V * E)
```

## Topological Sorting

Linear ordering of vertices in a Directed Acyclic Graph (DAG) such that for every edge  $u \rightarrow v$ ,  $u$  comes before  $v$ . Used in task scheduling, build systems, course prerequisites.

```
# Kahn's Algorithm (BFS-based)
from collections import deque
def topological_sort(V, adj):
    in_degree = [0] * V
    for u in range(V):
        for v in adj[u]:
            in_degree[v] += 1

    queue = deque([i for i in range(V) if in_degree[i] == 0])
    result = []

    while queue:
        u = queue.popleft()
        result.append(u)
        for v in adj[u]:
            in_degree[v] -= 1
            if in_degree[v] == 0:
                queue.append(v)

    if len(result) != V:
        return None # cycle exists
    return result
```

## Minimum Spanning Tree

An MST connects all vertices with minimum total edge weight, with no cycles.

Kruskal's Algorithm: Sort edges, add if no cycle (uses Union-Find)

Prim's Algorithm: Grow MST from a starting vertex (uses priority queue)

## 14. Sorting Algorithms

### Comparison of Sorting Algorithms

| Algorithm      | Best         | Average      | Worst        | Space      |
|----------------|--------------|--------------|--------------|------------|
| Bubble Sort    | $O(n)$       | $O(n^2)$     | $O(n^2)$     | $O(1)$     |
| Selection Sort | $O(n^2)$     | $O(n^2)$     | $O(n^2)$     | $O(1)$     |
| Insertion Sort | $O(n)$       | $O(n^2)$     | $O(n^2)$     | $O(1)$     |
| Merge Sort     | $O(n \lg n)$ | $O(n \lg n)$ | $O(n \lg n)$ | $O(n)$     |
| Quick Sort     | $O(n \lg n)$ | $O(n \lg n)$ | $O(n^2)$     | $O(\lg n)$ |
| Heap Sort      | $O(n \lg n)$ | $O(n \lg n)$ | $O(n \lg n)$ | $O(1)$     |
| Counting Sort  | $O(n+k)$     | $O(n+k)$     | $O(n+k)$     | $O(k)$     |
| Radix Sort     | $O(nk)$      | $O(nk)$      | $O(nk)$      | $O(n+k)$   |

### Bubble Sort

Repeatedly swaps adjacent elements if they are in wrong order. The largest element 'bubbles up' to the end in each pass.

```
Pass 1: [5,3,8,1,2] -> [3,5,1,2,8] (8 bubbled to end)
Pass 2: [3,5,1,2,8] -> [3,1,2,5,8] (5 bubbled)
Pass 3: [3,1,2,5,8] -> [1,2,3,5,8] (3 bubbled)
Pass 4: [1,2,3,5,8] -> [1,2,3,5,8] (no swaps = done!)
```

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if not swapped: # optimization
            break # already sorted
```

### Merge Sort (Divide & Conquer)

Divide array into halves, recursively sort each half, then merge. Guaranteed  $O(n \log n)$  but uses  $O(n)$  extra space.

```

[38, 27, 43, 3, 9, 82, 10]
    /           \
  [38, 27, 43, 3] [9, 82, 10]
    /   \       /   \
  [38, 27] [43, 3] [9, 82] [10]
    /  \  /  \   /  \  |
  [38] [27] [43] [3] [9] [82] [10]
    \  /  \  /   \  /   |
  [27, 38] [3, 43] [9, 82] [10]
    \       \       \   /
  [3, 27, 38, 43] [9, 10, 82]
        \       /
      [3, 9, 10, 27, 38, 43, 82]

```

```

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

```

## Quick Sort

Picks a pivot, partitions array so elements smaller than pivot are on the left, larger on the right, then recursively sorts both halves. Best and average case  $O(n \log n)$ .

```
def quick_sort(arr, low, high):
    if low < high:
        pivot_idx = partition(arr, low, high)
        quick_sort(arr, low, pivot_idx - 1)
        quick_sort(arr, pivot_idx + 1, high)

def partition(arr, low, high):
    pivot = arr[high] # last element as pivot
    i = low - 1 # index of smaller element
    for j in range(low, high):
        if arr[j] < pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i+1], arr[high] = arr[high], arr[i+1]
    return i + 1
```

```
Array: [10, 80, 30, 90, 40, 50, 70]
Pivot = 70
After partition: [10, 30, 40, 50, 70, 90, 80]
                  smaller ^ larger
                      pivot
Then recursively sort [10,30,40,50] and [90,80]
```

## Insertion Sort

Builds sorted array one element at a time. Picks each element and inserts it into its correct position in the already-sorted portion. Efficient for small or nearly sorted data.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j] # shift right
            j -= 1
        arr[j + 1] = key
    # Best:  $O(n)$  when already sorted
    # Worst:  $O(n^2)$  when reverse sorted
```

## 15. Searching Algorithms

### Linear Search

Simple search through each element. Works on both sorted and unsorted arrays.  $O(n)$  time.

```
def linear_search(arr, target):  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i  
    return -1
```

### Binary Search

Efficient search on sorted arrays. Repeatedly divides search space in half.

Time:  $O(\log n)$  - extremely efficient. Eliminates half the remaining elements each step.

```
Sorted array: [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]  
Target: 23
```

```
Step 1: mid = arr[4] = 16 < 23 -> search right  
Step 2: mid = arr[7] = 56 > 23 -> search left  
Step 3: mid = arr[5] = 23 = 23 -> FOUND at index 5!
```

```
10 elements searched in just 3 steps ( $\log_2(10) \sim 3.3$ )
```

```
# Iterative Binary Search
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

# Recursive Binary Search
def binary_search_rec(arr, target, low, high):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_rec(arr, target, mid+1, high)
    else:
        return binary_search_rec(arr, target, low, mid-1)
```

## Binary Search Variants

1. First Occurrence: Find the first position of target in a sorted array with duplicates
2. Last Occurrence: Find the last position of target
3. Search in Rotated Sorted Array: Array rotated at some pivot
4. Find Floor/Ceiling: Closest value  $\leq$  or  $\geq$  target
5. Square Root: Find integer square root using binary search

```
# First occurrence of target
def first_occurrence(arr, target):
    low, high = 0, len(arr) - 1
    result = -1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            result = mid
            high = mid - 1 # keep searching left
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return result

# Search in rotated sorted array
def search_rotated(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        # Left half is sorted
        if arr[low] <= arr[mid]:
            if arr[low] <= target < arr[mid]:
                high = mid - 1
            else:
                low = mid + 1
        else: # Right half is sorted
            if arr[mid] < target <= arr[high]:
                low = mid + 1
            else:
                high = mid - 1
    return -1
```

## 16. Dynamic Programming (DP)

Dynamic Programming is an algorithmic technique for solving problems by breaking them into overlapping subproblems and storing the results to avoid redundant computation. It applies when a problem has:

1. Optimal Substructure: Optimal solution contains optimal solutions to subproblems
2. Overlapping Subproblems: Same subproblems are solved repeatedly

### DP Approaches

1. Top-Down (Memoization): Start from main problem, recurse down, cache results
2. Bottom-Up (Tabulation): Start from smallest subproblems, build up to answer

### Example 1: Fibonacci Numbers

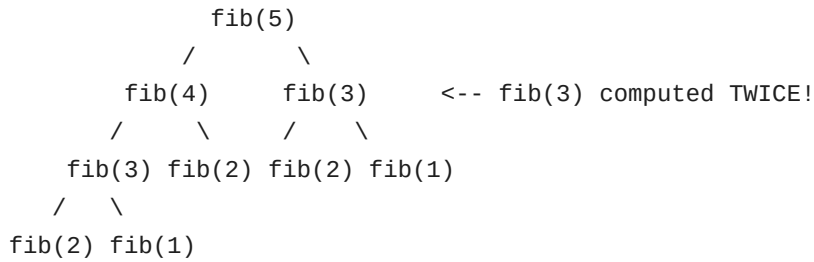
```
# Brute Force -  $O(2^n)$  - EXPONENTIAL
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2) # repeated work!

# Top-Down (Memoization) -  $O(n)$ 
def fib_memo(n, memo={}):
    if n in memo: return memo[n]
    if n <= 1: return n
    memo[n] = fib_memo(n-1) + fib_memo(n-2)
    return memo[n]

# Bottom-Up (Tabulation) -  $O(n)$ 
def fib_tab(n):
    if n <= 1: return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]

# Space Optimized -  $O(n)$  time,  $O(1)$  space
def fib_opt(n):
    if n <= 1: return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

Fib(5) Recursion Tree (without memoization):



With memoization, each fib(k) computed only once:  $O(n)$

## Example 2: 0/1 Knapsack Problem

Given items with weights and values, and a knapsack with capacity  $W$ , find the maximum value you can carry. Each item can be taken at most once.

```

def knapsack(weights, values, W):
    n = len(weights)
    # dp[i][w] = max value using first i items with capacity w
    dp = [[0] * (W + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(W + 1):
            # Don't take item i
            dp[i][w] = dp[i-1][w]
            # Take item i (if it fits)
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w],
                               dp[i-1][w - weights[i-1]] + values[i-1])
    return dp[n][W]

# Example:
# weights = [1, 3, 4, 5]
# values  = [1, 4, 5, 7]
# W = 7
# Answer: 9 (items with weight 3 and 4)

```

## Example 3: Longest Common Subsequence

```
def lcs(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]

# LCS of "ABCB DAB" and "BDCAB" = 4 (BCAB)
```

## Common DP Patterns

1. Fibonacci Pattern:  $dp[i] = dp[i-1] + dp[i-2]$
2. Knapsack Pattern:  $dp[i][w] = \max(\text{include}, \text{exclude})$
3. LCS Pattern: Match or skip characters
4. Matrix Chain:  $dp[i][j] = \min$  over all splits
5. LIS Pattern:  $dp[i] = \max$  length ending at  $i$
6. Coin Change:  $dp[\text{amount}] = \min$  coins to make amount
7. Grid Paths:  $dp[i][j] = \text{ways to reach } (i,j)$

## Example 4: Coin Change

```
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0 # 0 coins needed for amount 0

    for i in range(1, amount + 1):
        for coin in coins:
            if coin <= i and dp[i - coin] + 1 < dp[i]:
                dp[i] = dp[i - coin] + 1
    return dp[amount] if dp[amount] != float('inf') else -1

# coins = [1, 5, 10, 25], amount = 30
# Answer: 2 (25 + 5)
```

## 17. Greedy Algorithms

Greedy algorithms make the locally optimal choice at each step, hoping to find the global optimum. They work when the problem has:

1. Greedy Choice Property: A global optimum can be arrived at by selecting a local optimum
2. Optimal Substructure: An optimal solution to the problem contains optimal solutions to subproblems

### Greedy vs Dynamic Programming

| Feature   | Greedy             | Dynamic Programming  |
|-----------|--------------------|----------------------|
| Approach  | Local optimum      | All subproblems      |
| Choice    | One path           | Consider all options |
| Guarantee | Not always optimal | Always optimal       |
| Time      | Usually faster     | Usually slower       |
| Example   | Activity Selection | Knapsack             |

### Example: Activity Selection Problem

```
# Select maximum non-overlapping activities
def activity_selection(activities):
    # activities = [(start, end), ...]
    activities.sort(key=lambda x: x[1]) # sort by end time
    selected = [activities[0]]
    last_end = activities[0][1]

    for i in range(1, len(activities)):
        if activities[i][0] >= last_end: # no overlap
            selected.append(activities[i])
            last_end = activities[i][1]
    return selected

# Activities: [(1,4),(3,5),(0,6),(5,7),(3,9),(5,9),(6,10),(8,11)]
# Selected:  [(1,4),(5,7),(8,11)]
```

### Example: Fractional Knapsack

```
def fractional_knapsack(weights, values, W):
    n = len(weights)
    # Calculate value-to-weight ratio
    items = [(values[i]/weights[i], weights[i], values[i])
              for i in range(n)]
    items.sort(reverse=True) # sort by ratio

    total = 0
    for ratio, w, v in items:
        if w <= W:
            total += v
            W -= w
        else:
            total += v * (W / w) # take fraction
            break
    return total
```

## Other Greedy Problems

1. Job Sequencing: Sort by deadline, select profitable jobs
2. Huffman Coding: Build optimal prefix codes (compression)
3. Minimum Platforms: Sort arrivals and departures
4. Jump Game: Track the farthest reachable position
5. Gas Station: Find starting point for circular tour

## 18. Backtracking

Backtracking is a systematic method for exploring all possible solutions. It builds a solution incrementally and backtracks when it determines the current path cannot lead to a valid solution. It's essentially a depth-first search on a state-space tree.

### Backtracking Template

```
def backtrack(path, choices):
    if end_condition_met:
        result.append(path[:]) # record solution
        return
    for choice in choices:
        if not valid:
            continue # prune
        path.append(choice)    # choose
        backtrack(path, new_choices) # explore
        path.pop()            # un-choose (backtrack)
```

### Example 1: N-Queens Problem

Place N queens on an N×N chessboard such that no two queens attack each other (no two in same row, column, or diagonal).

4-Queens Solution:

```
. Q . .
. . . Q
Q . . .
. . Q .
```

Each queen is in a unique row, column, and diagonal.

```
def solve_n_queens(n):
    board = [['.' * n for _ in range(n)]]
    result = []

    def is_safe(row, col):
        for i in range(row):
            if board[i][col] == 'Q': return False
            if col - (row-i) >= 0 and board[i][col-(row-i)] == 'Q':
                return False
            if col + (row-i) < n and board[i][col+(row-i)] == 'Q':
                return False
        return True

    def backtrack(row):
        if row == n:
            result.append([''.join(r) for r in board])
            return
        for col in range(n):
            if is_safe(row, col):
                board[row][col] = 'Q'
                backtrack(row + 1)
                board[row][col] = '.' # backtrack

    backtrack(0)
    return result
```

## Example 2: Subset Sum Problem

```
def subset_sum(nums, target):
    result = []
    def backtrack(start, path, remaining):
        if remaining == 0:
            result.append(path[:])
            return
        if remaining < 0:
            return
        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1, path, remaining - nums[i])
            path.pop() # backtrack
    backtrack(0, [], target)
    return result

# nums = [2,3,6,7], target = 7
# Output: [[2, 5], [7]] (if 5 were in list)
```

## Example 3: Generate Permutations

```
def permute(nums):
    result = []
    def backtrack(path, remaining):
        if not remaining:
            result.append(path[:])
            return
        for i in range(len(remaining)):
            path.append(remaining[i])
            backtrack(path, remaining[:i] + remaining[i+1:])
            path.pop()
    backtrack([], nums)
    return result

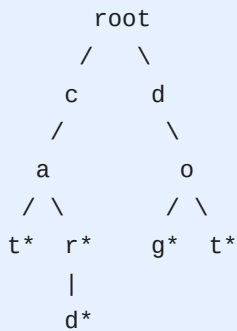
# nums = [1,2,3]
# Output: [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]
```

## 19. Trie (Prefix Tree)

A Trie is a tree-like data structure used for efficient retrieval of keys in a dataset of strings. Each node represents a character, and paths from root to marked nodes represent stored strings.

### Trie Structure

Insert: "cat", "car", "card", "dog", "dot"



\* = end of word marker

From root -> c -> a -> t = "cat"

From root -> c -> a -> r -> d = "card"

### Trie Implementation

```

class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end = True

    def search(self, word):
        node = self.root
        for char in word:
            if char not in node.children:
                return False
            node = node.children[char]
        return node.is_end

    def starts_with(self, prefix):
        node = self.root
        for char in prefix:
            if char not in node.children:
                return False
            node = node.children[char]
        return True

```

## Time Complexity

| Operation   | Time                           |
|-------------|--------------------------------|
| Insert      | $O(m)$ where $m$ = word length |
| Search      | $O(m)$                         |
| Starts With | $O(m)$                         |
| Delete      | $O(m)$                         |

## Applications

1. Autocomplete / Search suggestions
2. Spell checking
3. IP routing (longest prefix match)
4. Word games (Boggle, Scrabble)
5. Predictive text input

## 20. Union-Find (Disjoint Set Union)

Union-Find is a data structure that tracks a set of elements partitioned into disjoint (non-overlapping) subsets. It supports two operations:

1. Find: Determine which set an element belongs to
2. Union: Merge two sets into one

### How It Works

```
Initial: Each element is its own set
{0} {1} {2} {3} {4} {5}

Union(0,1): {0,1} {2} {3} {4} {5}
Union(2,3): {0,1} {2,3} {4} {5}
Union(0,2): {0,1,2,3} {4} {5}
Union(4,5): {0,1,2,3} {4,5}
Union(0,4): {0,1,2,3,4,5} (all connected!)
```

### Optimizations

1. Path Compression: During find, make each node point directly to root
2. Union by Rank: Attach smaller tree under larger tree root

With both optimizations: nearly  $O(1)$  per operation (amortized)

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x]) # path compression
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py: return False # already connected
        if self.rank[px] < self.rank[py]:
            px, py = py, px
        self.parent[py] = px # attach smaller under larger
        if self.rank[px] == self.rank[py]:
            self.rank[px] += 1
        return True

    def connected(self, x, y):
        return self.find(x) == self.find(y)
```

## Applications

1. Kruskal's Minimum Spanning Tree algorithm
2. Detecting cycles in undirected graphs
3. Finding connected components
4. Network connectivity problems
5. Image processing (connected pixels)

## 21. Practice Problems Collection

Here is a curated collection of practice problems organized by topic. Start with Easy problems and progress to Hard ones. These are common in coding interviews.

### Arrays

#### Problem A1: Two Sum

Easy

Given an array of integers `nums` and an integer `target`, return indices of the two numbers that add up to `target`. Each input has exactly one solution. Use hash map for  $O(n)$  solution.

Hint: T

#### Problem A2: Best Time to Buy and Sell Stock

Easy

Given `prices[i]` is the price of a stock on day `i`. Find the maximum profit from buying and selling once. You must buy before selling.

Hint: T

#### Problem A3: Product of Array Except Self

Medium

Given an array, return an array where each element is the product of all other elements. Do not use division. Solve in  $O(n)$  without using the output array as extra space.

Hint: U

#### Problem A4: Container With Most Water

Medium

Given `n` non-negative integers `a1, a2, ..., an` representing container heights, find two lines that together with the x-axis form a container that holds the most water.

Hint: T

#### Problem A5: Trapping Rain Water

Hard

Given `n` non-negative integers representing an elevation map, compute how much water it can trap after raining.

Hint: F

#### Problem A6: Maximum Subarray (Kadane's)

Medium

Find the contiguous subarray with the largest sum and return its sum.

Hint: d

### Linked Lists

#### Problem L1: Reverse a Linked List

Easy

Reverse a singly linked list iteratively and recursively.

Hint: U

**Problem L2: Merge Two Sorted Lists**

Easy

Merge two sorted linked lists into one sorted list.

Hint: C

**Problem L3: Detect Cycle**

Easy

Determine if a linked list has a cycle using  $O(1)$  space.

Hint: F

**Problem L4: Remove Nth Node From End**

Medium

Remove the  $n$ th node from the end of a linked list in one pass.

Hint: U

**Problem L5: Merge K Sorted Lists**

Hard

Merge  $k$  sorted linked lists into one sorted list efficiently.

Hint: U

## Stacks & Queues

**Problem S1: Valid Parentheses**

Easy

Given a string containing just '(', ')', '{', '}', '[' and ']', determine if the input is valid.

Hint: P

**Problem S2: Min Stack**

Medium

Design a stack that supports push, pop, top, and retrieving the minimum element in  $O(1)$  time.

Hint: U

**Problem S3: Daily Temperatures**

Medium

Given daily temperatures, find how many days you have to wait for a warmer temperature.

Hint: U

**Problem S4: Evaluate Reverse Polish Notation**

Medium

Evaluate the value of an arithmetic expression in Reverse Polish Notation (postfix).

Hint: P

## Trees

**Problem T1: Maximum Depth of Binary Tree**

Easy

Find the maximum depth (height) of a binary tree.

Hint: re

**Problem T2: Validate BST**

Medium

Check if a binary tree is a valid Binary Search Tree.

*Hint: P***Problem T3: Lowest Common Ancestor**

Medium

Find the lowest common ancestor of two nodes in a binary tree.

*Hint: I***Problem T4: Serialize and Deserialize Binary Tree**

Hard

Design an algorithm to serialize a binary tree to a string and deserialize it back.

*Hint: U***Problem T5: Binary Tree Level Order Traversal**

Medium

Return the level order traversal of a binary tree's nodes' values (left to right, level by level).

*Hint: U*

## Graphs

**Problem G1: Number of Islands**

Medium

Given a 2D grid of '1's (land) and '0's (water), count the number of islands.

*Hint: B***Problem G2: Clone Graph**

Medium

Deep copy a connected undirected graph.

*Hint: B***Problem G3: Course Schedule (Topological Sort)**

Medium

Determine if you can finish all courses given prerequisites (detect cycle in directed graph).

*Hint: K***Problem G4: Word Ladder**

Hard

Find the shortest transformation sequence from beginWord to endWord, changing one letter at a time.

*Hint: B***Problem G5: Network Delay Time**

Medium

Find the time it takes for all nodes to receive a signal from a source node.

*Hint: R*

## Dynamic Programming

**Problem D1: Climbing Stairs**

Easy

You can climb 1 or 2 steps. How many distinct ways to reach the  $n$ th step?

*Hint: It***Problem D2: Longest Increasing Subsequence**

Medium

Find the length of the longest strictly increasing subsequence.

*Hint: d***Problem D3: Coin Change**

Medium

Find the fewest coins needed to make up an amount. Return -1 if not possible.

*Hint: d***Problem D4: Longest Palindromic Substring**

Medium

Find the longest palindromic substring (not subsequence).

*Hint: E***Problem D5: Edit Distance**

Medium

Find the minimum number of operations (insert, delete, replace) to convert word1 to word2.

*Hint: d***Problem D6: House Robber**

Medium

Rob houses in a line for max money. Adjacent houses trigger alarm.

*Hint: d*

## Quick Reference: Problem Patterns

| Pattern        | When to Use                | Example Problems              |
|----------------|----------------------------|-------------------------------|
| Two Pointer    | Sorted array, pairs        | Two Sum, Container Water      |
| Sliding Window | Contiguous subarray        | Max Subarray Sum, Min Window  |
| Hash Map       | Frequency, lookup          | Two Sum, Group Anagrams       |
| Binary Search  | Sorted, search space       | Rotated Array, Sqrt           |
| BFS            | Level order, shortest path | Word Ladder, Islands          |
| DFS            | Exhaustive, path finding   | Permutations, N-Queens        |
| DP             | Overlapping subproblems    | Knapsack, LCS, LIS            |
| Greedy         | Local optimal              | Activity Selection, Huffman   |
| Stack          | Parentheses, next greater  | Valid Parentheses, Min Stack  |
| Heap           | Top K, median              | Top K Elements, Merge K Lists |
| Union-Find     | Connected components       | Redundant Connection          |
| Trie           | String prefix search       | Autocomplete, Word Search     |

# Appendix: How to Approach DSA Problems

---

## Step-by-Step Problem Solving

1. READ: Understand the problem completely. What are the inputs? What's the expected output?
2. EXAMPLES: Work through 2-3 examples by hand, including edge cases.
3. BRUTE FORCE: Come up with a naive solution. This helps understand the problem.
4. OPTIMIZE: Can you eliminate unnecessary work? Use the right data structure.
5. CODE: Write clean, readable code. Use meaningful variable names.
6. TEST: Walk through your code with your examples. Check edge cases.
7. ANALYZE: State the time and space complexity.

## Common Interview Tips

1. Think out loud - explain your approach before coding.
2. Ask clarifying questions about constraints and edge cases.
3. Start with the brute force solution, then optimize.
4. If stuck, try the brute force first - partial credit is better than nothing.
5. Use edge cases: empty input, single element, duplicates, negative numbers.
6. Test your code mentally with small examples.
7. Be ready to discuss time and space complexity.
8. Practice consistently - solve 2-3 problems daily.

## Recommended Learning Path

Week 1-2: Arrays, Strings, Time Complexity

Week 3-4: Linked Lists, Stacks, Queues

Week 5-6: Trees, BST, Heaps

Week 7-8: Hashing, Graphs (BFS, DFS)

Week 9-10: Sorting, Searching

Week 11-12: Dynamic Programming basics

Week 13-14: Advanced DP, Greedy, Backtracking

Week 15+: Practice interview problems, mock interviews

## Resources for Further Learning

- LeetCode: [leetcode.com](https://leetcode.com) (problem practice)
- NeetCode 150: [neetcode.io](https://neetcode.io) (curated problem list)
- Striver's SDE Sheet: [takeuforward.org](https://takeuforward.org) (structured preparation)
- CLRS: Introduction to Algorithms (textbook)
- Visualgo.net: Visualize algorithms
- Big O Cheat Sheet: [bigoocheatsheet.com](https://bigoocheatsheet.com)